

Gerichtete Graphen

$$G = (V, E), \quad E \subseteq \{(u, v) \mid u, v \in V, u \neq v\}$$

$\deg_{\text{in}}(v)$ = Anzahl eingehender Kanten

$\deg_{\text{out}}(v)$ = Anzahl ausgehender Kanten

Quelle falls $\deg_{\text{in}}(v) = 0$



Senke falls $\deg_{\text{out}}(v) = 0$



Weg, Zyklus, Pfad, Kreis analog wie bei ungerichteten Graphen, mit der zusätzlichen Bedingung, dass man nur in Richtung der Kante gehen darf

Wie viele gerichtete Pfade von e nach c?

Wie viele gerichtete Pfade von c nach e?

Wie viele gerichtete Wege von c nach e?

Wie viele gerichtete Kreise?

Was sind alle möglichen Längen gerichteter Wege von d nach e?

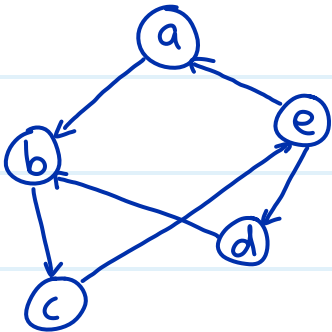
gerichteter Hamiltonpfad?

gerichteter Hamiltonkreis?

zusammenhängend?

gibt es einen gerichteten Kreis der a und d enthält?

gibt es einen gerichteten Zyklus der a und d enthält?



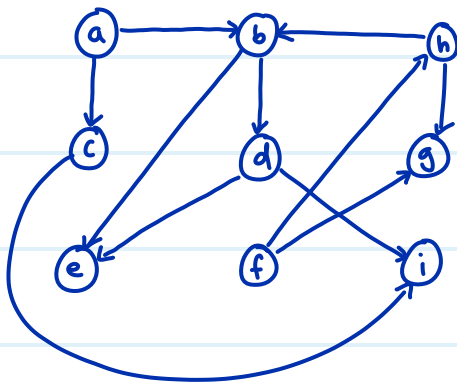
$\exists$ back edge $\Leftrightarrow \exists$ gerichteter Zyklus

$\nexists$ back edge $\Rightarrow$ umgekehrte Post-order ist eine topologische Sortierung

Laufzeit: Adjazenzmatrix: $O(n^2)$

Adjazenzliste: $O(n+m)$ $\Rightarrow$ nimmt immer Adjazenzlisten!

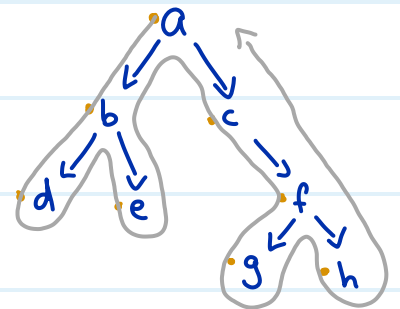
DFS mit Startknoten a, bei mehreren Möglichkeiten alphabetisch



Trick für pre/post-order:

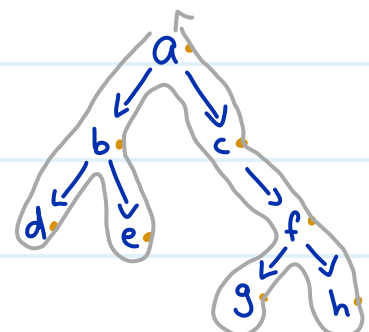
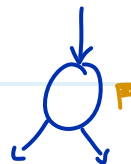
DFS-Baum ablaufen

Flagge links
= Pre



a, b, d, e, c, f, g, h

Flagge rechts
= Post

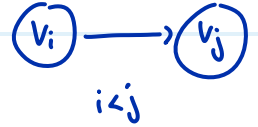


d, e, b, g, h, f, c, a

Exercise 9.4 Longest path in a DAG (1 point). Bonus HS24

Let $G = (V, E)$ be a directed graph without directed cycles (i.e., a directed acyclic graph or short DAG). Assume that $V = \{v_1, \dots, v_n\}$ (for $n = |V| \in \mathbb{N}$) and that the sorting $v_1, v_2, \dots, v_n$ of the vertices is a topological sorting. The goal of this exercise is to find the longest path in G .

(a) Let P be a path in G . Prove that if $P = (v_{i_1}, v_{i_2}, \dots, v_{i_k})$, then $i_1 < i_2 < \dots < i_k$.



(b) Describe a bottom-up dynamic programming algorithm that, given a graph G with the property that $v_1, \dots, v_n$ is a topological sorting, returns the length of the longest path in G in $O(|V| + |E|)$ time. You can assume that the graph is provided to you as a pair (n, Adj) of the integer $n = |V|$ and the adjacency lists Adj . Your algorithm can access $Adj[u]$, which is a list of vertices to which u has a direct edge, in constant time. Formally, $Adj[u] := \{v \in V \mid (u, v) \in E\}$.

In your solution, address the following aspects:

1. *Dimensions of the DP table:* What are the dimensions of the DP table?
2. *Subproblems:* What is the meaning of each entry?
3. *Recursion:* How can an entry of the table be computed from previous entries? Justify why your recurrence relation is correct. Specify the base cases of the recursion, i.e., the cases that do not depend on others.
4. *Calculation order:* In which order can entries be computed so that values needed for each entry have been determined in previous steps? Describe the calculation order in pseudocode.
5. *Extracting the solution:* How can the solution be extracted once the table has been filled?
6. *Running time:* What is the running time of your solution?

Hint: Define the entry of the DP table as $DP[i] = \text{length of longest path in } G \text{ ending at vertex } v_i$.

Dimension :

Teilproblem :

Base Case :

Rekursion :

Berechnungsreihenfolge:

Lösung:

Laufzeit:

Write a Java program that counts the number of **connected components** in an undirected graph represented as an adjacency list.

= Anzahl Bäume im DFS Wald

